MarQira Pulse — Dashboard & Site-Detail Redesign

Branch: `main` **Commit:** `d0f03fa` — Redesign dashboard & site-detail UI to new design spec with real data **Previous HEAD:** `d59f925` **Push status:** ✅ Pushed to `origin/main` (verified via `git ls-remote`: `d0f03fab… = refs/heads/main`). **Frontend build:** ✅ `tsc -b && vite build` — 158 modules, 0 TypeScript errors. **Backend tests:** ✅ `php vendor/bin/pest` (non-parallel) — **218 passed, 660 assertions**. **Database / schema changes: NONE.** Every new metric is derived from data the connector already collects (`site_heartbeats`, `sites.plugin_version`, `sites.enrolled_at`/`created_at`, `core_update_available`, `plugin_updates_count`, `theme_updates_count`, existing visitor/user/content snapshots). No migrations were added.

The two uploaded HTML files (`marqira-pulse-dashboard.html`, `marqira-pulse-site-detail.html`) were treated as a **design specification**, not static markup. They were translated into the existing React/Vite/TypeScript/Tailwind component architecture using reusable components. Every placeholder metric, random chart series, and hard-coded value in the mockups was replaced with **real data** from the API (or an honest empty/no-data state where the account genuinely has no data yet).

---

1. Major UI changes (system-wide)

- **New design-token system.** `tailwind.config.js` and `src/index.css` now define semantic token families backed by CSS variables — `surface` (DEFAULT/soft/pale/grid), `ink` (DEFAULT/soft/body/muted), `line` (DEFAULT/strong), `nav`, plus brand/indigo/sky/success/warning/danger accents and the `brand-`

`gradient`. Typography uses Inter (sans), Space Grotesk (display), JetBrains Mono (mono).

- **Full light/dark theming.** `ThemeContext` (`src/context/ThemeContext.tsx`) resolves saved preference → OS preference → light, sets `data-theme` on `<html>`, and persists to `localStorage` (`mqp-theme`). Every page was migrated off hard-coded `slate-*`/`bg-white`/`red-*`/`emerald-*` utility classes onto the tokens, so both themes render correctly everywhere (pages, modals, tables, auth screens).
- **Reusable component library.** Shared primitives were consolidated:
  - `components/ui.tsx` — `Spinner, LoadingState, ErrorState, EmptyState, StatusBadge, Badge, VerifiedPill, FavAvatar, Pill, SiteStatusPill`.
  - `components/charts.tsx` — `CountUp, Sparkline, AreaChart, Seg, RolloutRing, PulseLine` (all pure SVG, no chart library, null-safe on empty data).
  - `components/Topbar.tsx`, `components/AccountSelector.tsx`, `components/ThemeToggle.tsx`, and a token-migrated `components/Layout.tsx` / `components/Modal.tsx`.
- **Consistent states.** Every data surface has explicit loading, error (with retry), and empty (“No … yet”) states rather than blank space or fabricated numbers.
- **Responsive.** Grids collapse to single-column on narrow viewports; tables use horizontal-scroll containers only where unavoidable; the layout targets laptop down to small screens.

---

2. Main Dashboard (Overview) changes

Implemented in `src/pages/Overview.tsx`.

- **Pulse hero** with the decorative animated `PulseLine`.
- **Four metric cards** using `CountUp`, driven by a new backend `trends` block:
  - Total websites + sites added this month.
  - 7-day fleet uptime %.
  - Pending updates, broken down into core / plugins / themes.
  - Attention count.

- **Fleet uptime card** with a `Seg` range switch (7 / 30 / 90 days) that calls `GET /api/dashboard/fleet/uptime` and renders a real `AreaChart` (domain 0–100).
- **Role-aware panel:** owners see a live **Activity feed** (`/audit-logs`); subscribers see a scoped **Attention** card (they never receive org-wide audit data).
- **Update queue** and **Connector release** cards from real data.
- **Quick actions**.

---

3. Site Details changes

Implemented in `src/pages/WebsiteDetail.tsx` — a full rewrite into a tabbed layout with a site header (favicon tile, status pill, multisite pill, meta line), a 5-item quick-stat strip, and an underline tab-bar. All tabs use **real API data**:

Tab	Data source	Notes
Overview	site detail payload	Identity / Infrastructure / Software / Health info-cards
Traffic	`GET /sites/{uuid}/visitors?days=…`	Real visitor series + `AreaChart` + daily breakdown table + 7/30 range switch
Users	site users payload	Totals, admins (from roles), logged-in-in-7d (from `last_login_at`), paginated table
Content	content summary payload	Type tiles + filter chips + paginated table

Tab	Data source	Notes
WordPress	site detail	Core/version/theme info-cards
Plugin status	connector health + heartbeat cadence	Real reporting cadence
Network	site detail	Low-confidence note + **preserved** manual verify form
Connection history	heartbeat history	Real heartbeat table
Updates	real update counters	Summary tiles + connector self-update banner + **preserved** WordPress maintenance actions (core/plugins/themes) + live command status
Activity	`/audit-logs?subject_uuid=…`	Site-scoped timeline

Preserved functionality not in the mockup: the manual network/ownership verification form, all WordPress remote-maintenance actions, live remote-command status, and the connector self-update flow.

---

4. New features introduced from the designs

- **Fleet uptime analytics** (Overview chart + `trends.uptime_7d_pct`).

- **Connector rollout view** (`src/pages/PluginReleases.tsx`): a `RolloutRing` donut plus a "This release" card showing how many sites are on the latest connector version, how many are behind, and how many are not reporting — from `GET /api/dashboard/fleet/rollout`.
- **This-month growth / update breakdown trends** on the Overview metric cards.
- **Light/dark appearance picker** in Settings, wired to `ThemeContext`.
- **Verified-origin pills** and **real visitor sparklines** in the Websites list.

---

5. Previously-missing features that required backend work

The designs asked for fleet-wide uptime and connector-rollout figures that the API did not previously expose. New backend code was added (no schema change):

- **`app/Services/FleetAnalytics.php`** — `uptime(array $siteIds, Collection $sites, int $range): array` returns `{ series, has_data, average }`. Daily availability = distinct sites with ≥1 heartbeat that day ÷ sites enrolled on/before that day. Driver-aware (Postgres/SQLite date handling) and empty-safe (null percentage when no sites were enrolled yet).
- **`app/Http/Controllers/Api/Dashboard/FleetController.php`**
  - `uptime(Request)` — validates `range ∈ {7,30,90}` (default 7).
  - `rollout(Request)` — groups sites by `plugin_version` (null/empty → not reporting), marks the active `PluginRelease::getActive()?->version`, and sorts by `version_compare` descending.
  - Both build their site set through `ScopesToAccount::scopedSitesQuery()` + `$this->tenantContext`, so they are strictly tenant-scoped.
- **`app/Http/Controllers/Api/Dashboard/OverviewController.php`** — added a `trends` block: `{ sites_added_this_month, uptime_7d_pct, updates_breakdown: { core, plugins, themes } }`.

- **`routes/api.php`** — added `/fleet/uptime` and `/fleet/rollout` inside the main authenticated tenant/dashboard group (deliberately **not** owner-only, so subscribers get their own scoped fleet view).

---

6. New / modified APIs

Method	Endpoint	Purpose
GET	`/api/dashboard/fleet/uptime?range=7|30|90`	Fleet uptime series (new)
GET	`/api/dashboard/fleet/rollout`	Connector-version rollout distribution (new)
GET	`/api/dashboard/overview`	Now includes a `trends` block (modified)

No existing endpoints changed shape in a breaking way; the overview change is additive.

---

7. New analytics / telemetry

No new telemetry collection was added on the connector side — all new analytics are **computed from data already collected**:

- **Fleet uptime** is computed from existing `site_heartbeats` rows against each site's enrollment date.
- **Rollout distribution** is computed from the `plugin_version` each site already reports on heartbeat.
- **Growth / update breakdown** trends are computed from existing site columns.

---

8. How each major new metric gets real data

- **7-/30-/90-day fleet uptime** — `FleetAnalytics::uptime()` counts, per day, the distinct scoped sites that produced at least one heartbeat, divided by the sites enrolled by that day.
- **Sites added this month** — count of scoped sites whose `created_at`/`enrolled_at` falls in the current month.
- **Pending updates (core/plugins/themes)** — summed from `core_update_available`, `plugin_updates_count`, `theme_updates_count` across scoped sites.
- **Connector rollout** — `FleetController::rollout()` groups scoped sites by reported `plugin_version` and compares against the active release version.
- **Per-site traffic / users / content** — the existing site visitor, user, and content snapshot endpoints, filtered to the one authorized site.

---

9. Intentional design deviations (and why)

- **Per-site “7-day uptime” sparkline in the Websites table — omitted.** The `sites` list payload has no per-site uptime series; only the fleet-level series exists. Rather than fabricate a per-row trend, the column was left out and uptime is shown at the fleet level on Overview. (Adding a per-site series would require a new aggregated endpoint; flagged as a future enhancement.)
- **Updates tab per-plugin update list (WooCommerce/Yoast/etc.) — not implemented as shown.** The mockup listed named per-plugin updates, but the backend exposes update **counts**, not a per-plugin update manifest. Real update-count summary tiles + the real WordPress maintenance actions were kept instead. Honest and functional rather than fabricated.
- **All Math.random / hard-coded arrays from the mockups’ inline JS (traffic generator, sample posts, sample users, sample connection times, and the “Illustrative traffic**

data" note) were dropped and replaced with live API data or empty states.
- **`design/` reference HTML was intentionally not committed** — it is a working baseline snapshot, not part of the application.

---

10. Multi-tenant isolation verification

The prior visitor-leak class of bug was specifically guarded:

- Every fleet/overview query resolves its site set through **`ScopesToAccount::scopedSitesQuery(Request)`**, which is `Site::where(organization_id)->visibleTo($user)->active()` and applies the owner-only account filter **fail-closed**. A normal subscriber can only ever receive their own sites' data; the selected account/site context governs what is displayed.
- Owners retain cross-account visibility, but the selected user/site context still scopes the response.
- Per-site tabs fetch by the site UUID, which is itself resolved through the scoped query, so one account cannot read another account's site by guessing a UUID.
- The isolation is enforced in the **backend queries**, not the frontend.
- Covered by the existing `VisitorIsolationTest` and the rest of the 218-test suite, all green.

---

11. Tests performed & results

- **Frontend:** `cd apps/dashboard && npm run build` → `tsc -b && vite build` completed with **0 errors**, 158 modules, CSS ~41.6 kB / JS ~389 kB.
- **Backend:** `cd apps/api && php vendor/bin/pest` (non-parallel) → **218 passed (660 assertions)**, including the 9 new `FleetAnalyticsTest` cases and the multi-tenant `VisitorIsolationTest`.
- **Manual code sweep:** grepped the entire `pages/` + `components/` tree for leftover legacy color classes and mock artifacts

(`Math.random`, `console.log`, “illustrative”, fake data) — **none remain**.

---

12. Design features that could not be fully implemented

- **Per-site uptime sparkline in the Websites table** — blocked by the absence of a per-site uptime time series in the current data model (only fleet-level aggregation exists). Would require a new aggregation endpoint; documented above as a deviation.
- **Named per-plugin update list in the Updates tab** — blocked by the connector reporting update counts rather than a per-plugin update manifest. Documented above.

Both are data-availability limitations, not UI limitations; everything the current data supports is implemented with real values.

Notes

- Local dev uses a near-empty SQLite database (few sites, no heartbeats), so empty/no-data states are the expected view locally; production Postgres has real fleet data that populates every chart and metric.
- If any private-repo access is ever missing for automation, grant the **Abacus GitHub App** access to the repository.